



EL PROBLEMA DETRÁS DE ESTE PROYECTO

Hasta el 40 % de las cosechas

se pierde cada año por **enfermedades en los cultivos.**

El diagnóstico temprano es la clave, pero requiere expertos escasos.

¿Y si bastara una foto de la hoja para diagnosticar?



UNIVERSIDAD
NACIONAL
DE COLOMBIA

CLASIFICACIÓN DE ENFERMEDADES EN PLANTAS MEDIANTE VISIÓN POR COMPUTADOR

MOBILENETV2, EXPORTACIÓN CON EXECUTORCH Y DESPLIEGUE EN HUGGING
FACE SPACES

Jabes Esteban Monroy Becerra

Universidad Nacional de Colombia

Procesamiento Digital de Imágenes | Junio 2026

CONTEXTO DEL PROBLEMA

UN PROBLEMA REAL DE LA AGRICULTURA

Por qué importa:

- Las enfermedades de cultivos amenazan la **seguridad alimentaria** y causan grandes pérdidas económicas.
- El diagnóstico tradicional es **manual, lento** y depende de expertos no disponibles en zonas rurales.
- Un diagnóstico **temprano y automático** permite actuar a tiempo y reducir el uso de agroquímicos.

Solución: una herramienta que, a partir de la foto de una hoja, identifique el estado de salud del cultivo.

Indicador	Valor
Cultivos cubiertos	14
Clases (cultivo+estado)	38
Imágenes	~54.000
Accuracy en test	99,4 %

Referencia: Mohanty et al. (2016) validaron el uso de deep learning para diagnóstico de enfermedades de plantas por imagen.

Tarea: clasificación de imágenes (multiclase, 38 clases).

- Entrada: foto de una hoja.
- Salida: cultivo + estado (sano / enfermedad).

¿Por qué clasificación y no detección/segmentación? La hoja ocupa toda la imagen; no se necesita localizar ni delimitar por píxel, solo **categorizar**.

Base de datos: PlantVillage

- Fuente pública (Kaggle), variante a color.
- ~54.000 imágenes, 38 clases.
- Formato carpeta por clase (*ImageFolder*).

Partición	Proporción
Entrenamiento	70 %
Validación	15 %
Prueba	15 %

Partición estratificada + *data augmentation* en entrenamiento.

El contexto (Hughes & Salathé, 2015):

- Para 2050 habrá que producir **~70 % más alimento** (población >9 mil millones).
- Las enfermedades reducen el rendimiento un **40 % en promedio** (hasta 100 % en países en desarrollo).
- Los **smartphones** masivos permiten un diagnóstico móvil accesible mediante *machine learning*.
- **PlantVillage** libera más de **50.000 imágenes curadas** de hojas sanas e infectadas.

“Estos datos son el inicio de un esfuerzo de crowdsourcing para que la visión por computador ayude a resolver las pérdidas de cosecha por enfermedades.”

Base reportada para la prueba en caliente: cualquier imagen de PlantVillage (color) puede subirse al Space.

EXPLORACIÓN DEL DATASET: COMPOSICIÓN

14 CULTIVOS, 38 CLASES

Cultivo	Clases	Enferm.
Tomate	10	9
Manzana	4	3
Maíz	4	3
Uva	4	3
Papa	3	2
Cereza	2	1
Durazno	2	1
Pimentón	2	1
Fresa	2	1
Naranja	1	1
Calabaza	1	1
Arándano	1	0
Frambuesa	1	0
Soya	1	0
Total	38	26

Lectura de la composición:

- **38 clases** = 12 sanas + 26 enfermedades.
- **Tomate** concentra 10 clases; varios cultivos tienen una sola.
- Esto produce **desbalance entre clases**, que motiva usar **Macro-F1** además del accuracy.
- Imágenes en **condiciones controladas** (hoja sobre fondo uniforme).

Mitigación: *data augmentation* (volteos, rotación, jitter) para mejorar la generalización.

VARIABLES DEL DATASET Y SUS TIPOS

QUÉ REPRESENTA CADA DATO Y CÓMO SE MODELA

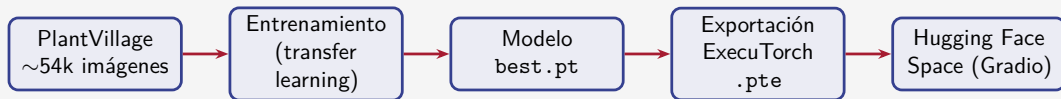
Variable	Tipo	Descripción
Imagen (entrada)	Imagen RGB (JPEG)	Foto de la hoja, $\sim 256 \times 256$ px; se redimensiona a 224×224 .
Canales de color	3 enteros (0–255)	R, G, B; se normalizan con la media/desv. de ImageNet.
Clase (salida)	Categórica nominal, 38 niveles	Cultivo + estado; se codifica como índice entero 0–37.
Cultivo	Categórica, 14 niveles	Derivada del nombre de la clase (Tomate, Manzana, ...).
Estado	Categórica binaria	Sano vs. enfermedad.

Nota: no hay variables tabulares ni valores faltantes. Cada muestra es un par (**imagen, clase**): la imagen es la única *feature* de entrada y la clase es la variable objetivo que el modelo predice.

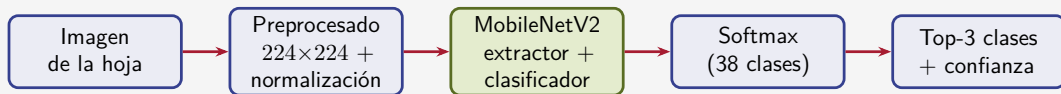
FLUJO DEL PROYECTO Y DEL MODELO

CÓMO SE CONSTRUYE Y CÓMO ACTÚA SOBRE UNA IMAGEN

1. Flujo del proyecto (entrenamiento y despliegue):



2. Flujo de inferencia (cómo actúa el modelo):



El modelo recibe la foto, la lleva a 224x224 normalizada, extrae características con MobileNetV2 y produce una distribución de probabilidad sobre las 38 clases.

Modelo: MobileNetV2 (Sandler et al., 2018) preentrenado en ImageNet (Deng et al., 2009), con *transfer learning*.

Por qué esta arquitectura:

- **Ligera y eficiente** (bloques *inverted residual*) → inferencia rápida en CPU.
- Ideal para **despliegue en la nube/edge** → encaja con ExecuTorch.
- El *transfer learning* reutiliza características visuales ya aprendidas → **alta precisión con menos datos y épocas**.

Componente	Elección
Backbone	MobileNetV2 (ImageNet)
Cabeza	Lineal a 38 clases
Optimizador	Adam ($\text{lr} = 10^{-3}$)
Entrada	224×224 , normalizada

Solo se reemplaza la capa final; el resto se afina sobre PlantVillage.

Función de pérdida:

Entropía cruzada con *label smoothing*

(Szegedy et al., 2016).

- Estándar para **clasificación multiclase**: mide la divergencia entre la distribución predicha y la real.
- El *label smoothing* evita el **exceso de confianza**, regulariza y mejora la generalización.

Nota: por eso la pérdida no baja a 0, sino que se estabiliza ($\approx 0,70$). Es esperado.

Métricas:

Accuracy (principal) + Macro-F1.

- *Accuracy*: desempeño global, intuitiva.
- *Macro-F1*: promedia el F1 **por clase con igual peso**, así detecta fallos en clases minoritarias.

Clave: PlantVillage está **desbalanceado**; la Macro-F1 evita que un alto accuracy oculte clases mal clasificadas.

EVOLUCIÓN DEL MODELO POR ÉPOCA

REGISTRO DEL ENTRENAMIENTO

```
*** Usando dispositivo: cuda
38 clases detectadas
Downloading: "https://download.pytorch.org/models/mobilenet_v2-b0353104.pth" to /root/.cache/torch/hub/checkpoints/mobilenet_v2-b0353104.pth
100% 13.6M/13.6M [00:00<00:00, 145MB/s]
Época 01/10 | loss 0.9250/0.8046 | acc 0.9382/0.9697
-> Mejor modelo guardado (val_acc=0.9697)
Época 02/10 | loss 0.7849/0.7482 | acc 0.9772/0.9844
-> Mejor modelo guardado (val_acc=0.9844)
Época 03/10 | loss 0.7566/0.7712 | acc 0.9834/0.9768
Época 04/10 | loss 0.7581/0.7345 | acc 0.9842/0.9812
Época 05/10 | loss 0.7422/0.7569 | acc 0.9853/0.9767
Época 06/10 | loss 0.7352/0.7214 | acc 0.9868/0.9914
-> Mejor modelo guardado (val_acc=0.9914)
Época 07/10 | loss 0.7301/0.7230 | acc 0.9877/0.9866
Época 08/10 | loss 0.7254/0.7192 | acc 0.9888/0.9889
Época 09/10 | loss 0.7266/0.7225 | acc 0.9878/0.9890
Época 10/10 | loss 0.7175/0.7026 | acc 0.9982/0.9929
-> Mejor modelo guardado (val_acc=0.9929)

Test | loss 0.7034 | acc 0.9926 | macro-F1 0.9920
Artefactos guardados en artifacts
```

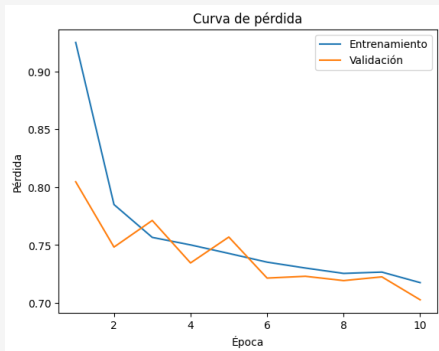
Pérdida y accuracy (entrenamiento/validación) reportados en cada época.

Lectura:

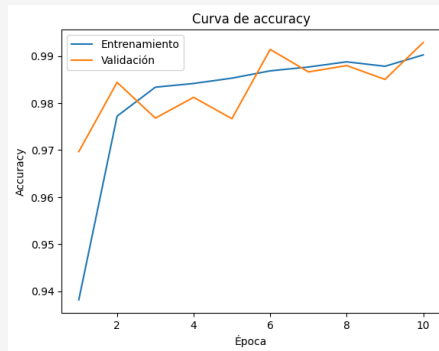
- El accuracy de validación pasa de **97,6 %** (época 1) a **99,3 %** (mejor época).
- La pérdida desciende de forma sostenida en ambas particiones.
- Se guarda automáticamente el **mejor modelo** según el accuracy de validación.

GRÁFICAS DE ENTRENAMIENTO

EVOLUCIÓN DEL MODELO A LO LARGO DE LAS ÉPOCAS



Pérdida por época (entrenamiento y validación).



Accuracy por época (entrenamiento y validación).

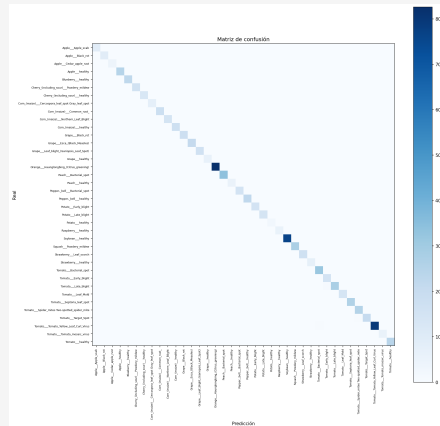
Lectura: la pérdida de entrenamiento y validación bajan **juntas** (sin sobreajuste) y el accuracy se estabiliza en **~99 %** en ambas particiones. La validación va pareja al entrenamiento, lo que confirma **buena generalización**.

RESULTADOS FINALES

DESEMPEÑO SOBRE EL CONJUNTO DE PRUEBA INDEPENDIENTE

Métrica	Valor
Accuracy (test)	99,39 %
Macro-F1 (test)	99,15 %
Mejor accuracy (val.)	99,30 %

Lectura: la diagonal fuerte de la matriz de confusión indica aciertos **consistentes en las 38 clases**; los pocos errores se dan entre enfermedades visualmente similares.



Matriz de confusión (38 clases) sobre el conjunto de prueba.

DESPLIEGUE: EXECUTORCH Y HUGGING FACE SPACES

DEL MODELO ENTRENADO A UNA DEMO EN LA NUBE

Flujo de exportación:

`best.pt` → `torch.export` → ExecuTorch

$\xrightarrow{\text{XNNPACK}}$ `model.pte`

- **ExecuTorch** compila el modelo a un formato eficiente para CPU.
- El `.pte` predice **idéntico** al modelo original (diferencia $\sim 4 \times 10^{-6}$).
- Desplegado con **Gradio** en Hugging Face Spaces.

Interfaz pública (en la nube):

Componente	Tecnología
Modelo	<code>model.pte</code> (ExecuTorch)
Runtime	ExecuTorch XNNPACK
Interfaz	Gradio
Hosting	Hugging Face Spaces

El usuario sube una foto y obtiene el diagnóstico (top-3 con confianza) en segundos.

DEMO Y PRUEBA EN CALIENTE

INFERENCIA EN TIEMPO REAL SOBRE UNA IMAGEN CUALQUIERA

Space desplegado y público:

[huggingface.co/spaces/
jabesmonroy/plant-disease-classifier](https://huggingface.co/spaces/jabesmonroy/plant-disease-classifier)

Cómo funciona la prueba en vivo:

- 1 Se toma una imagen cualquiera del dataset.
- 2 Se sube directamente al Space.
- 3 El modelo predice la clase en tiempo real.



Interfaz del Space: Apple___Apple_scab con 97 % de confianza. Incluye un modo de varias imágenes.

CONCLUSIONES

SOLUCIÓN COMPLETA, EFICIENTE Y DESPLEGADA

- **MobileNetV2 con transfer learning** clasifica 38 estados de cultivos con **99,4 % de accuracy** en test.
- La **entropía cruzada con label smoothing** y las métricas **accuracy + Macro-F1** son adecuadas para una tarea multiclase y desbalanceada.
- Las curvas confirman aprendizaje **sin sobreajuste** y buena generalización (validación pareja al entrenamiento).
- El modelo se exportó con **ExecuTorch** y se desplegó en **Hugging Face Spaces**, listo para inferencia en tiempo real.

De una foto de hoja a un diagnóstico, en segundos y en la nube.

- Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). *Using Deep Learning for Image-Based Plant Disease Detection*. *Frontiers in Plant Science*, 7, 1419.
- Hughes, D. P., & Salathé, M. (2015). *An open access repository of images on plant health* (PlantVillage). arXiv:1511.08060.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. CVPR.
- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). *Rethinking the Inception Architecture for Computer Vision*. CVPR.
- Deng, J., et al. (2009). *ImageNet: A Large-Scale Hierarchical Image Database*. CVPR.
- PyTorch Edge. *ExecuTorch Documentation*. pytorch.org/exectuorch